PRAMANA : Citation-Verified AI Legal Research for Indian Criminal Law

A Project Report

Submitted by

SUVAN SHARMA (2328069)
Team: The Spartan Coders

Under the guidance of

Aditya Sharma

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

At



I.K. GUJRAL PUNJAB TECHNICAL UNIVERSITY
MOHALI CAMPUS-1

CERTIFICATE

This is to certify that the project report titled **"PRAMANA"** has been successfully completed and submitted by:

- **Suvan Sharma (2328069)**

in partial fulfillment of the requirements for the degree of **Bachelor of Technology (B.Tech)** in **Computer Science & Engineering** at **I.K. Gujral Punjab Technical University, Mohali Campus 1**.

This project has been carried out under the guidance and supervision of **Aditya Sharma** and is an authentic record of the work done by the student during the academic session **2026**.

We hereby certify that the matter presented in this report is original and has not been submitted elsewhere for any other degree or diploma.

Signature of Student

Suvan Sharma (2328069) - _____

Signature of Mentor

Aditya Sharma

[Designation]

I.K. Gujral Punjab Technical University, Mohali Campus 1

Date:

ACKNOWLEDGEMENT

I take this opportunity to express my deepest gratitude to all those who have supported and guided me throughout the completion of my project titled

"PRAMANA: CITATION-VERIFIED AI LEGAL RESEARCH FOR INDIAN CRIMINAL LAW."

I am sincerely thankful to **Aditya Sharma**, my project guide, for continuous support, motivation and valuable insights that helped shape this project from concept to reality.

I would also like to thank the faculty and staff of **I.K. GUJRAL PUNJAB TECHNICAL UNIVERSITY, MOHALI CAMPUS-1**, for providing the necessary resources and environment to carry out this work smoothly.

I also acknowledge the public resources this project is built on: the Government of India's publication of the Indian Penal Code and the Bharatiya Nyaya Sanhita, the open archive of Supreme Court judgments, and the authors of the NyayaAnumana and IL-TUR datasets.

Lastly, I extend my heartfelt thanks to my family and friends for their constant encouragement, understanding and moral support during the course of this project.

Suvan Sharma (2328069)

TABLE OF CONTENTS

Sr. no.	Content	Page no.
1.	Introduction	7-8
2.	Problem statement	9-10
3.	Literature review	11-12
4.	Objectives	13
5.	Methodology	14-15
6.	System architecture	16-17
7.	Technology Stack	18-19
8.	Data Ingestion & Knowledge Base Module	20-21
9.	Offence-Date Router & Retrieval Module	22
10.	Citation Graph & Verification Module	23
11.	Multi-Agent Reasoning Pipeline	24-25
12.	Data Flow Diagram	26-27
13.	Use Case Diagram	28-29
14.	Implementation Details	30-32
15.	Testing & Evaluation	33-34
16.	Results	35-36
17.	Limitations	37
18.	Future Scope	38
19.	Conclusion	39
20.	References	40

ABSTRACT

On 1 July 2024 India replaced the Indian Penal Code, 1860 (IPC) with the Bharatiya Nyaya Sanhita, 2023 (BNS). Almost every offence was renumbered, while offences committed before that date continue to be tried under the IPC, so for years lawyers must work with both codes. The same section number now denotes unrelated offences in the two codes: IPC 302 is murder while BNS 302 punishes wounding religious feelings, BNS 303 is theft, and the BNS has no section 420 at all. At the same time, large language models (LLMs) used for legal research are known to fabricate citations, and the transition multiplies the opportunity for such errors.

Pramana is an AI-assisted legal research system for Indian criminal law built around one principle: *the language model proposes, deterministic code decides what is real*. A lawyer enters the facts of a case and the offence date. A deterministic offence-date router selects the applicable code; a retrieval stage fuses statute similarity (BGE-M3 embeddings in ChromaDB) with votes from similar past judgments using reciprocal rank fusion; a local LLM proposes candidate sections; and a section is admitted into the report only if it exists as a node of the applicable code in a NetworkX citation graph built from the ingested statutes and judgments. Rejected proposals are fed back to the model with the reason through a bounded retry loop.

The pipeline is implemented as a multi-agent LangGraph state machine. After verification, an applicability judge tests each verified section against the facts using verbatim quotes and deterministic legal rules; a precedent matcher retrieves graph-linked judgments, bridging new BNS charges to their IPC predecessors through a reviewed 575-row mapping; an outcome analyst summarises how appeals on each section were decided using plain SQL; and a defence drafter writes counter-arguments that are re-verified before the report is shown. Results are delivered through a FastAPI backend and a web interface that streams each stage live.

The knowledge base holds all 575 IPC sections, all 358 BNS sections parsed from the official gazette, 2,699 Supreme Court and High Court judgments (87,036 indexed passages) and a citation graph of 933 section nodes and 16,493 citation edges. In every evaluation no non-existent or wrong-code section reached a report: the citation gate separated 20 of 20 real and fabricated section numbers, routing was correct in 30 of 30 runs, and when the model cited from memory the verifier caught 20 of its 68 citations and every affected case recovered within two retries. On the IL-TUR statute-identification benchmark the system, without any task-specific training, reaches macro-F1 11.8 on the test split against 28.1 for the best supervised model; choosing the right section, rather than inventing one, remains the main area for improvement.

The project is organised as independent modules (src/ingest, src/retrieval, src/graph, src/pipeline, src/api), each covered by automated tests (422 in total), so that every component can be verified and extended on its own.

1. INTRODUCTION

1.1 Background and Motivation

Legal research is the first step of every criminal matter. A lawyer reads the facts, identifies the offences they disclose, finds the statutory text and locates precedents in which courts applied those sections. In India this work is still largely manual: candidate sections are found in the bare act, their ingredients are read in commentaries, and precedent is located through keyword search over judgment databases followed by hours of reading.

Since 1 July 2024 this work has become harder. The Bharatiya Nyaya Sanhita, 2023 replaced the Indian Penal Code, 1860 for offences committed on or after that date, and almost every offence received a new number. Because an offence committed earlier is still tried under the IPC, both codes will be in active use for years, and a section number is meaningless without its code. The code, in turn, is decided by the date of the offence.

Generative AI promises faster research, but general-purpose chatbots and even commercial legal research tools have been shown to produce confident, fabricated citations. During development the language model used in this project was asked what BNS 303 covers; it described a bomb-threat offence. BNS 303 is theft. A lawyer relying on such an answer would cite a section that does not say what the brief claims.

Pramana addresses this specific and checkable failure structurally rather than statistically. The applicable code is chosen by the offence date, never by the model; every section the model proposes is looked up in a citation graph built from the actual statutory text of that code; and mechanical legal conditions, such as five persons for an unlawful assembly or a death for homicide, are enforced by rules. The language model is used only where language understanding is unavoidable, and each of its outputs is checked again by code.

1.2 The Pramana System

Pramana is a complete, end-to-end legal research pipeline. It comprises five subsystems that work in concert:

- **Knowledge base:** the IPC and BNS statutes and 2,699 judgments, cleaned, chunked and embedded into four ChromaDB collections, together with a NetworkX citation graph and an SQLite outcomes table.
- **Offence-date router and retrieval:** a deterministic rule that selects the IPC or the BNS, and a retrieval stage that ranks candidate sections from statute text and from the sections cited by similar judgments.
- **Multi-agent reasoning pipeline:** a LangGraph state machine in which agents propose, verify, test, ground and argue, all reading and writing one shared, typed state.
- **Verification layer:** a code-scoped citation check with a bounded retry loop, and a final pass that re-checks every section, precedent and quotation used in the report.

- **Web interface:** a FastAPI backend and a browser interface that shows each stage live and presents the report, with tools such as the bare act, an IPC–BNS comparison and a research memo.

1.3 Key Features

- **Date-based code routing:** the offence date decides the code; ambiguous dates are never guessed, and an offence period that crosses 1 July 2024 is analysed under both codes.
- **Every citation verified:** a section is shown only if it exists in the citation graph of the routed code.
- **Applicability with reasons:** each real section is marked supported, doubted (with the missing ingredient) or included in a greater charge.
- **Precedent across both codes:** BNS charges reach IPC case law through a reviewed IPC→BNS mapping.
- **Grounded counter-arguments and outcome statistics:** arguments quote the facts verbatim; appeal outcomes are counted, never generated.
- **Lawyer tools:** bare-act text, a word-level IPC↔BNS comparison, punishment and bail from the official First Schedules, PDF upload of an FIR or order, follow-up questions and a printable research memo.

1.4 Scope

The system covers substantive criminal law under the IPC and the BNS. It accepts a description of facts in English (or a digital PDF of an FIR or court order) and an offence date, and produces a research report. It is a research aid for advocates, law students and legal-aid clinics; it is not legal advice and does not replace a lawyer's judgment.

1.5 Three-Step Usage Workflow

- **Step 1 – Describe the case:** the user types or pastes the facts; the offence date is read from the text, and the interface asks for it when there is none or more than one.
- **Step 2 – Watch the analysis:** the pipeline view shows each stage as it finishes: the code chosen, the candidate sections with their scores, each proposal and its verification, and the applicability verdicts.
- **Step 3 – Read the report:** a result summary lists the charges as supported, doubted or included, followed by precedents, counter-arguments and appeal outcomes, with tools to open the bare act, compare codes, ask questions and download a memo.

2. PROBLEM STATEMENT

2.1 The Renumbering of Indian Criminal Law

The BNS re-enacts most IPC offences under new numbers, merges the IPC's separate definition and punishment sections for many offences (IPC 378 defines theft and IPC 379 punishes it, while BNS 303 does both), moves some provisions into sub-sections (common intention, IPC 34, is now BNS 3(5)) and drops others (adultery, IPC 497). Table 1 shows how the same numbers now mean different things.

Table 1: The same section number in the two codes

Section	IPC, 1860 (offences before 1 July 2024)	BNS, 2023 (offences from 1 July 2024)
302	Punishment for murder	Uttering words with deliberate intent to wound religious feelings
303	Punishment for murder by a life-convict	Theft
420	Cheating and dishonestly inducing delivery of property	No such section (the BNS ends at section 358)

A citation that is correct in one code is therefore a different offence, or nothing at all, in the other. Any tool that does not first decide the code from the offence date, and then check citations against that code only, will produce citations that look right and are wrong.

2.2 Hallucinated Citations in AI Legal Tools

Large language models generate fluent text from patterns, not from a database of law. When asked about a section, they can produce a plausible but invented answer. Measured on this project's 15 development cases with retrieval withheld, the model cited 68 sections from memory, and 20 of them did not exist in the BNS, mostly old IPC numbers used in BNS-era cases. Published studies report the same failure in commercial legal research tools.

2.3 Limitations of Existing Research Methods

- **Manual research is slow:** finding sections, reading their ingredients and locating precedent takes hours per matter.
- **Search engines do not interpret facts:** legal databases return documents for keywords; they do not map a description of facts to the sections it discloses.
- **AI assistants can invent law:** without a structural check, an answer may cite a section that does not exist or belongs to the other code.
- **Precedent is split across codes:** new BNS sections have almost no case law yet, so precedent must be found under their IPC predecessors.

2.4 Formal Problem Statement

To build a research assistant for Indian criminal law that, given the facts of a case and the offence date, identifies the applicable sections and precedents across the IPC→BNS

transition, generates likely counter-arguments and section-level outcome statistics, and never presents a citation that does not exist in the applicable code.

3. LITERATURE REVIEW

3.1 Indian Legal NLP and Statute Identification

- **LeSICiN (Paul et al., AAAI 2022):** a heterogeneous graph model that links fact descriptions, sections and citations for automatic statute identification. It is supervised and trained on IPC-era data.
- **InLegalBERT (Paul et al., ICAIL 2023):** a BERT model pre-trained on Indian legal text; strong for supervised tasks after fine-tuning.
- **IL-TUR (Joshi et al., ACL 2024):** a benchmark of Indian legal understanding tasks, including Legal Statute Identification (LSI) over 100 IPC labels; the best published LSI macro-F1 is 28.1 (LeSICiN).
- **ILSIC (Paul et al., EACL 2026):** statute identification from layperson queries; zero-shot GPT-4.1 reached 31.6 micro-F1 and 14.4 macro-F1, and the retriever was identified as the bottleneck.
- **NyayaAnumana (Nigam et al., COLING 2025):** the largest Indian judgment-prediction dataset, labelled with whether an appeal or petition succeeded; a filtered subset forms this project's High Court corpus.

3.2 Retrieval-Augmented Generation

Retrieval-augmented generation (Lewis et al., NeurIPS 2020) grounds a model's output in retrieved documents rather than in its parameters. It keeps knowledge current, because new documents can be added without retraining, and it makes answers traceable to their sources. RAG on its own, however, does not check that a cited section exists; that check is the contribution of this project's verification layer.

3.3 Embeddings and Rank Fusion

- **BGE-M3 (Chen et al., 2024):** a multilingual, multi-granularity embedding model producing 1024-dimensional vectors.
- **LLM-based embedders for prior case retrieval (Premasiri et al., RANLP 2025):** general-purpose embedders were found competitive with, or better than, legal-domain models on Indian prior case retrieval, a result this project reproduced in its own bake-off.
- **Reciprocal rank fusion (Cormack et al., SIGIR 2009):** combines several rankings by summing $1/(k + \text{rank})$; used here with $k = 60$ to fuse statute similarity with precedent votes.

3.4 Hallucination in Legal AI

- **Dahl et al. (Journal of Legal Analysis, 2024):** measured frequent legal hallucinations in general-purpose chatbots.
- **Magesh et al. (2024):** found that commercial RAG-based legal research tools still hallucinate on a substantial share of queries.

These studies measure the failure; they do not prevent it. Pramana addresses one specific, checkable class of hallucination, the existence of a citation in the applicable code, structurally, and treats applicability as a separate, softer problem.

3.5 Existing Tools

- **Indian Kanoon:** a widely used search engine for Indian case law and statutes; it returns documents for a query but does not interpret a set of facts.
- **Manupatra and SCC Online:** subscription legal databases with extensive case law and editorial content; they are search and reading tools rather than fact-to-section reasoning systems.
- **jhana.ai and NyayAssist:** AI legal assistants for Indian law that attach and check citations in their answers.

3.6 Research Gap

Existing systems are either supervised models trained for one task on the IPC only, search and database tools that do not interpret facts, or AI assistants whose answers are not routed by offence date. No existing system combines **date-based code routing**, **graph-verified citations**, an **IPC→BNS precedent bridge**, **grounded counter-arguments** and **section-level outcome statistics** in one pipeline. Pramana is designed to fill this gap.

4. OBJECTIVES

Pramana is developed with a defined set of objectives spanning the technical, legal and user-experience dimensions of the problem.

4.1 Primary Technical Objectives

- **Correct code routing:** select the IPC or the BNS from the offence date with a deterministic rule, correct in every test case.
- **Section identification:** identify the sections a set of facts discloses, using retrieval-augmented generation.
- **Citation verification:** never present a section that does not exist in the applicable code, by checking every proposal against a citation graph and retrying rejected proposals with the reason.
- **Precedent retrieval across codes:** find precedent judgments for each charge, including new BNS sections through their IPC predecessors.
- **Grounded generation:** draft counter-arguments that quote the facts verbatim and cite only supplied precedents, and verify them before display.

4.2 Secondary Operational Objectives

- **Applicability reasoning:** distinguish sections that are supported by the facts from those that are doubted, with the reason shown.
- **Outcome analytics:** report how appeals on each section were decided, computed by SQL from the judgments, with small samples suppressed.
- **Live, explainable interface:** show each stage of the analysis as it happens, and keep verified, doubted and rejected sections visually distinct.
- **Reproducible evaluation:** measure every claim with a script and saved per-case records, on development, held-out, real court-order and benchmark data.
- **Modularity and testability:** structure the system as independent modules covered by automated tests.

5. METHODOLOGY

The development of Pramana followed an incremental, gate-driven approach. The work was divided into seven phases, each with a written and measurable completion criterion (a "gate") and its own version-control branch, merged only when the gate was met. Every method change was measured against the same code with only that change switched off (ablation switches), and then scored once on a held-out set written before the change.

5.1 Phase 1 – Data Layer (2 weeks)

The IPC was taken from a structured JSON source and completed from the official consolidated text where sections were truncated; the BNS was parsed from the official Gazette of India PDF because no reliable machine-readable copy existed. Supreme Court judgments for 2023–2026 and a filtered High Court subset were ingested, cleaned and chunked: statutes one section per chunk, judgments by paragraph, never by a fixed token count. Every chunk carries metadata (source, court, date, sections involved). A 575-row IPC→BNS mapping was proposed by a similarity scorer and every row reviewed against the gazette text. **Gate:** the processed data validates against a fixed schema, and 20 random chunks were read and found intact.

5.2 Phase 2 – Retrieval (1 week)

All chunks were embedded into four ChromaDB collections. The embedding model was chosen by a measured bake-off on 15 hand-written cases: BGE-M3 reached recall@3 0.533 against 0.200 for the legal-domain InLegal-Sbert. Statute retrieval was fused with precedent votes by reciprocal rank fusion. **Gate:** relevant sections in the top 3 for at least 12 of 15 test cases, with a held-out set scored once.

5.3 Phase 3 – Citation Graph (1 week)

A NetworkX graph of section and judgment nodes, citation edges and verified IPC→BNS mapping edges was built, with verify_section(), verify_case() and get_precedents_for_section(). **Gate:** 100% correct on an adversarial set of 10 real and 10 fabricated section numbers (20/20 achieved).

5.4 Phase 4 – Single-Node Pipeline (2 weeks)

The LangGraph state machine, the offence-date router, the insufficient-data guardrail, Node 1 (statute identifier), Node 2 (verifier) and the conditional retry edge were implemented. **Gate:** 10 test cases (5 IPC-era, 5 BNS-era) routed correctly every time over three repetitions (30/30), and an adversarial case demonstrably triggered and survived the retry loop.

5.5 Phase 5 – Full Pipeline (2 weeks)

The precedent matcher with the IPC-predecessor bridge, the defence drafter with its final verification pass, the applicability judge with deterministic legal rules, and the outcome analyst (an offline extraction job into SQLite, with SQL-only aggregation at run time) were added, with precedents and outcomes running in parallel. **Gate:** the motorcycle-theft worked example runs end to end and produces a correctly structured report.

5.6 Phase 6 – Evaluation (1 week)

The system was evaluated on hand-written sets, on real High Court orders from an external dataset, and on the IL-TUR statute-identification benchmark, with the hallucination catch rate computed from every proposal the model made. **Gate:** reproducible numbers in a results document, produced by re-running one script.

5.7 Phase 7 – Interface (1 week)

A FastAPI backend wrapping the full pipeline and a browser interface were built, with live stage-by-stage progress and a report in which verified and doubted sections are visually distinct. **Gate:** the worked example runs end to end in the browser with no manual steps.

6. SYSTEM ARCHITECTURE

Overview

Pramana follows a modular, layered architecture with an offline part, built once, and a runtime pipeline executed for every case (Figure 1). The guiding rule is that the language model only proposes, judges and drafts; everything that decides whether a citation is real is deterministic code or a graph lookup.

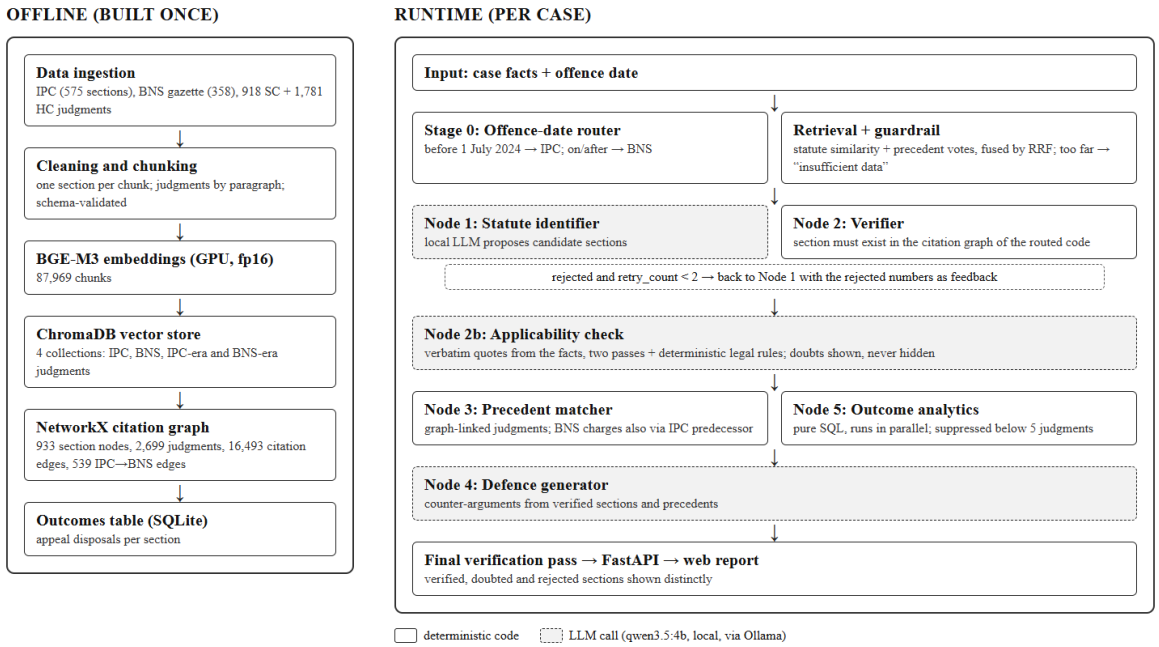


Figure 1: System architecture of Pramana

Architecture Components

1. Knowledge Base (offline): statutes and judgments are cleaned, chunked and embedded into ChromaDB; the citation graph and the outcomes table are built from the same data.

2. Offence-Date Router (Stage 0): a plain function that maps an offence date before 1 July 2024 to the IPC and any later date to the BNS. Every later lookup uses only this code.

3. Retrieval and Guardrail: the facts are embedded and matched against statute text; similar judgments vote for the sections they applied; the two rankings are fused. If the closest judgment is below a calibrated similarity threshold, the pipeline stops early and returns an "insufficient data" response instead of asking the model.

4. Multi-Agent Pipeline: the statute identifier proposes sections; the verifier checks each against the graph and sends rejections back; the applicability judge tests each verified section; the precedent matcher and the outcome analyst run in parallel; the defence drafter writes counter-arguments.

5. Final Verification and Report: every section, precedent label and quotation in the generated text is re-checked; the report is served through FastAPI to the web interface.

Working Flow of the System

1. The user enters the facts and the offence date.
2. The router selects the IPC or the BNS from the date.
3. Retrieval ranks about 14 candidate sections; the guardrail checks that the facts are close to the corpus.
4. The statute identifier proposes sections from the candidates.
5. The verifier confirms each proposal in the citation graph; rejected proposals return to the identifier with the reason, at most twice.
6. The applicability judge marks each verified section supported, doubted or included in a greater charge.
7. The precedent matcher and the outcome analyst run in parallel.
8. The defence drafter writes counter-arguments, which pass a final check.
9. The report is displayed, with verified, doubted and rejected sections kept visually distinct.

Key Features of the Architecture

- Separation of proposal (language model) from decision (code and graph).
- A single shared, typed state, so every step's output can be checked independently.
- Bounded retries and an explicit insufficient-data path, so the system never forces an answer from weak context.
- Modular components that can be replaced or improved individually.

7. TECHNOLOGY STACK – DEEP DIVE

Overview

Pramana is built from mature open-source components spanning natural language processing, information retrieval, graph computation and web development. Table 2 summarises the stack; each choice is explained below.

Table 2: Technology stack

Layer	Technology	Purpose
Language	Python 3.12	all components
Embeddings	BAAI/bge-m3 (sentence-transformers, PyTorch)	1024-dimensional vectors for facts, sections and judgments
Vector store	ChromaDB	four persistent collections with metadata filters
Citation graph	NetworkX	existence check, precedents, IPC→BNS bridge
Orchestration	LangGraph	multi-agent state machine with a retry edge and parallel nodes
Language model	qwen3.5:4b served by Ollama	proposes sections, judges applicability, drafts arguments
Outcomes	SQLite	appeal-disposal counts per section
PDF processing	PyMuPDF	statute gazette, judgments, uploaded FIRs and orders
Backend / interface	FastAPI + HTML/JavaScript web app	API, live progress, report and lawyer tools
Testing	pytest	422 automated tests

1. Python (Core Programming Language)

Python is used throughout for ingestion, retrieval, the agent pipeline and the API. Its ecosystem (PyTorch, sentence-transformers, NetworkX, FastAPI) covers every layer of the system, and its readability keeps the modules easy to test.

2. BGE-M3 (Embedding Model)

BGE-M3 converts text into 1024-dimensional vectors. It was selected over the legal-domain InLegal-Sbert by measurement: on 15 hand-written cases it reached recall@3 0.533 against 0.200, and the domain model showed "hubness", returning the same generic sections for almost every query. Indexing runs in half precision on the GPU.

3. ChromaDB (Vector Store)

ChromaDB stores the embeddings in four collections: IPC sections, BNS sections, IPC-era judgments (83,918 passages) and BNS-era judgments (3,118 passages). It is persistent on disk and supports metadata filters, such as restricting a search to judgments that cite a given section.

4. NetworkX (Citation Graph)

The citation graph holds every section of both codes as a node, every judgment as a node, an edge whenever a judgment cites a section, and a reviewed mapping edge from each IPC section to its BNS counterpart. A section is real only if it is a node of the routed code; this exact check is the foundation of the verification layer.

5. LangGraph (Multi-Agent Orchestration)

LangGraph runs the agents as a state machine over one shared, typed state. It provides what a single large prompt cannot: a conditional edge that returns rejected proposals to the identifier, parallel branches for precedents and outcomes, and intermediate outputs that can each be verified.

6. qwen3.5:4b on Ollama (Language Model)

A 4-billion-parameter open-weight model, served by Ollama through its REST API, is used in three places: proposing sections, judging applicability and drafting counter-arguments. Its answers are constrained to a JSON schema with bounded lists, decoded at temperature 0 with a fixed seed.

7. SQLite, PyMuPDF and FastAPI

SQLite stores the appeal-outcome counts so that statistics are exact and never generated by a model. PyMuPDF extracts text from the BNS gazette, the judgment PDFs and uploaded documents. FastAPI exposes the pipeline as typed endpoints, including a job interface that streams each stage to the web interface.

8. DATA INGESTION & KNOWLEDGE BASE MODULE

Overview

The knowledge base is built once from public sources and validated before use. It supplies the statute text, the judgment passages, the citation graph and the outcomes table on which every runtime stage depends.

Table 3: Data sources

Source	Content	Size after processing
IPC, 1860	structured JSON, 80 sections completed from the official text	575 sections
BNS, 2023	parsed from the Gazette of India PDF	358 sections
Supreme Court judgments	2023–2026, criminal judgments applying a penal code	918 judgments
High Court judgments	NyayaAnumana subset: theft, hurt, cheating, breach of trust	1,781 judgments
IPC→BNS mapping	every row reviewed against the gazette text	575 rows, 539 bridges
Total indexed	judgment passages in ChromaDB	87,036 passages

8.1 Processing Steps

- **Parsing:** text is extracted from PDFs with PyMuPDF; ligature glyphs padded with spaces ("fi led" for "filed"), running page headers and margin letters are repaired or removed by rule.
- **Chunking:** statutes are chunked one section per chunk; judgments by paragraph, or by runs of whole sentences where the source has no paragraphs; oversized paragraphs are split at sentence boundaries.
- **Section extraction:** the sections each passage cites are extracted, and citations of other Acts that reuse the same numbers (for example the Code of Criminal Procedure) are filtered out by reading the words after each citation.
- **Routing by code:** judgments are stored in the IPC-era or BNS-era collection according to the code they apply, so each collection speaks one numbering.
- **Validation:** every record is validated against a fixed schema, and each output file is cross-checked against the build report's counts.
- **Embedding:** every chunk is embedded with BGE-M3 and written to its ChromaDB collection.

8.2 Data Quality Findings

Reading samples, rather than relying on automated validation alone, uncovered several defects that were fixed before use: ligature damage in 8.6% of judgment chunks, running headers in 17%, BNS marginal-note titles lost or misassigned, a single "paragraph" of

397,044 characters, and 7,035 Criminal Procedure Code citations stored as IPC sections. After the citation-extraction fix, a fresh audit of 100 random graph edges found 96% correct.

9. OFFENCE-DATE ROUTER & RETRIEVAL MODULE

Overview

This module decides which code applies and finds the candidate sections the agents will read.

9.1 Offence-Date Router

The router is a deterministic function: an offence date before 2024-07-01 selects the IPC; that date or later selects the BNS. Dates are parsed strictly, because 08/09/2024 is a different day in India and the United States and a guessed date is a guessed code; future dates are rejected. An offence period that crosses the transition is analysed under both codes with a warning. In the web interface the date is read from the text by regular expressions only, and the user is asked when no single clear date is found.

9.2 Retrieval

Retrieval combines two signals. **Statute similarity** compares the embedded facts with section text, including index-only views of long sections split by sub-section. **Precedent votes** come from the judgments most similar to the facts: each votes for the sections it applied, weighted by its best similarity, and an IPC judgment can vote in a BNS-era case only through the verified mapping. The two rankings are fused by reciprocal rank fusion:

$$RRF(x) = 1 / (k + r_stat(x)) + 1 / (k + r_vote(x)), \quad k = 60$$

The identifier reads the fused top 10 followed by any new sections from the statute-only top 10, about 14 candidates, each with a short excerpt. For the motorcycle-theft example, BNS 303 (theft) ranks first with a fused score of 0.0320 (third by statute text, second by votes), followed by BNS 317 (stolen property) and BNS 324 (mischief).

9.3 Insufficient-Data Guardrail

If the best judgment similarity is below 0.576, the midpoint between the highest non-legal input and the lowest criminal fact pattern in a calibration set, the language model is not called and the nearest sections are returned for review. For the motorcycle example the closest judgment scored 0.686, so the agents were asked.

10. CITATION GRAPH & VERIFICATION MODULE

Overview

This module turns the language model's proposals into verified sections. It is the heart of the system's guarantee that no invented citation reaches a report.

10.1 The Citation Graph

The graph has 933 section nodes (all 575 IPC and all 358 BNS sections), 2,699 judgment nodes, 16,493 judgment-to-section citation edges and 539 IPC→BNS mapping edges. Its interface is verify_section(graph, code, section), verify_case(graph, id), get_precedents_for_section(graph, code, section) and the bridge functions get_ipc_predecessors() and get_bns_successor(). The code argument is mandatory: checking against the union of the two codes would accept, for example, "BNS 420", which does not exist.

10.2 Verification and the Retry Loop

Each proposal is parsed, recording any code it names, and looked up in the graph of the routed code. Rejections are classified (does not exist, belongs to the other code, not chargeable, another Act) and may carry a hint read from the graph. If any proposal is rejected and fewer than two retries have been used, the case returns to the identifier with its previous answer and the rejected numbers listed as banned. In a stress run with retrieval withheld, the model proposed 421 (rejected: the BNS ends at 358), then 378 (rejected: an IPC number whose BNS counterpart is 303), then 303, which was verified.

10.3 Chargeability

Definitions and general exceptions are real sections but never charges, so they are rejected as non-chargeable; a verified IPC definition is charged under the section that punishes it (for example 378 → 379).

11. MULTI-AGENT REASONING PIPELINE

Overview

The runtime pipeline is a set of agents built with LangGraph, all reading and writing one shared, typed state (Figure 2). Each agent writes only its own fields, so every step can be checked independently.

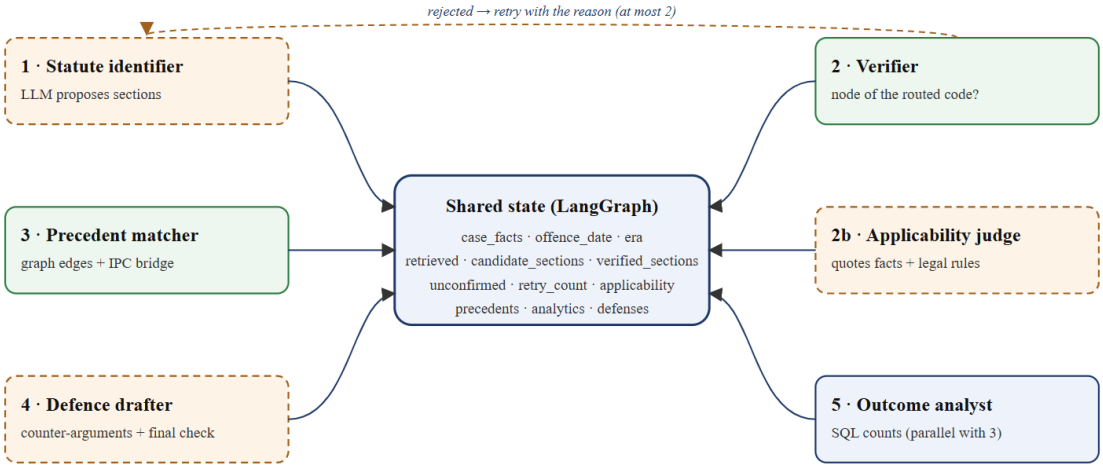


Figure 2: Six agents around the shared LangGraph state

Table 4: Agents and the state fields they write

Agent	Kind	Writes
1 · Statute identifier	Language model	candidate_sections (every round's proposals)
2 · Verifier	Graph lookup	verified_sections, unconfirmed (with reasons), retry_count
2b · Applicability judge	Language model + rules	applicability (supported / doubted / included)
3 · Precedent matcher	Graph lookup	precedents
5 · Outcome analyst	SQL	analytics
4 · Defence drafter	Language model + final check	defenses

11.1 Applicability Judge

Existing is not the same as applying. The judge reads each verified section's text against the facts, twice with the section order reversed, and must quote the facts verbatim for each ingredient. A section is supported only if both passes support it; otherwise it is shown as doubted with the missing ingredient. Deterministic rules override or supply judgments where the law is mechanical: unlawful assembly and rioting need five or more accused, homicide needs a reported death, an attempt to murder is never doubted for want of a death, the injury grade and weapon select the hurt section, and a plain offence beside a supported aggravated form is shown as included in it. In the knife-stabbing sample, attempt to murder and hurt by a dangerous weapon were supported, culpable homicide was doubted because no death occurred, and grievous hurt was shown as included in the weapon charge.

11.2 Precedent Matcher

Precedents come only from graph edges, so every precedent actually applied the charged section; they are ordered by similarity to the facts, and near-identical companion appeals are merged. For a BNS charge the matcher also follows the mapping to the IPC predecessor. This is necessary: BNS 303 has no direct precedent in the corpus, while its predecessors IPC 378 and IPC 379 have 11 and 158 citing judgments.

11.3 Outcome Analyst

The outcome analyst counts, with plain SQL, how appeals and petitions citing each section were disposed of, and suppresses sections with fewer than five judgments. For IPC 379, 158 judgments: 76 allowed, 76 dismissed, 1 partly allowed and 5 other. The figures are labelled as appeal disposals, not conviction rates.

11.4 Defence Drafter and Final Check

The drafter writes up to four counter-arguments from the verified charges, the statute text, the judge's doubts and the labelled precedents. A final deterministic pass checks that each argument targets a charged section, cites only supplied precedent labels that exist in the graph, and quotes the facts verbatim; any failure is flagged on the argument rather than hidden.

12. DATA FLOW DIAGRAM

Overview

The Data Flow Diagram (DFD) shows how data moves through Pramana: from the user's facts and date, through the processing stages and data stores, to the verified report.

Level 0 DFD (Context Diagram)

The Level 0 DFD presents the whole system as a single process. The user supplies case facts and an offence date and receives a verified report; the system draws on the statutes, the judgments and the IPC→BNS mapping, and calls the language model.

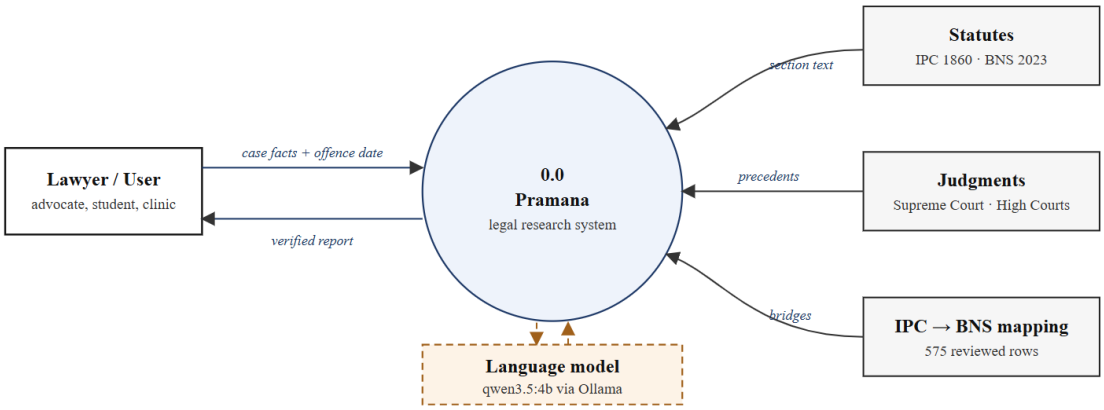


Figure 3: Level 0 DFD (context diagram)

Level 1 DFD (Detailed Diagram)

The Level 1 DFD breaks the system into its internal processes and data stores.

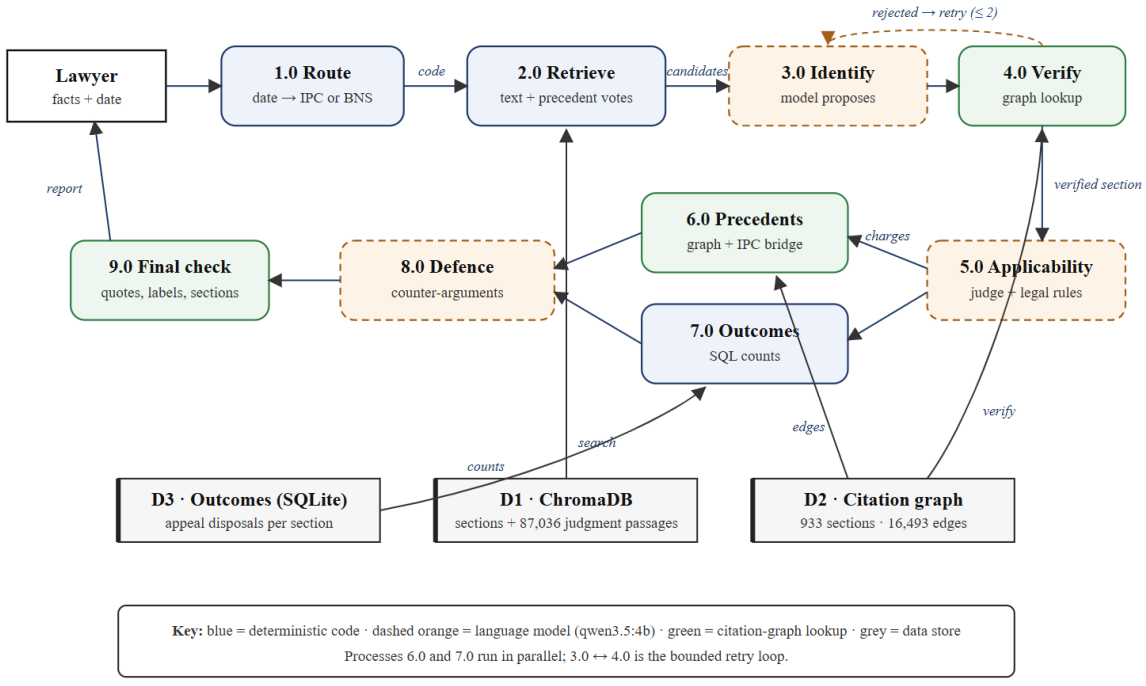


Figure 4: Level 1 DFD

Main Processes

- **1.0 Route:** reads the offence date and outputs the code.
- **2.0 Retrieve:** searches D1 and ranks candidate sections.
- **3.0 Identify and 4.0 Verify:** propose sections and confirm them in D2, with the bounded retry loop between them.
- **5.0 Applicability:** grades each verified section.
- **6.0 Precedents and 7.0 Outcomes:** read D2 and D3 in parallel.
- **8.0 Defence and 9.0 Final check:** draft and verify counter-arguments, then assemble the report.

13. USE CASE DIAGRAM

Overview

The Use Case Diagram shows the interaction between the actors and Pramana from a user perspective (Figure 5).

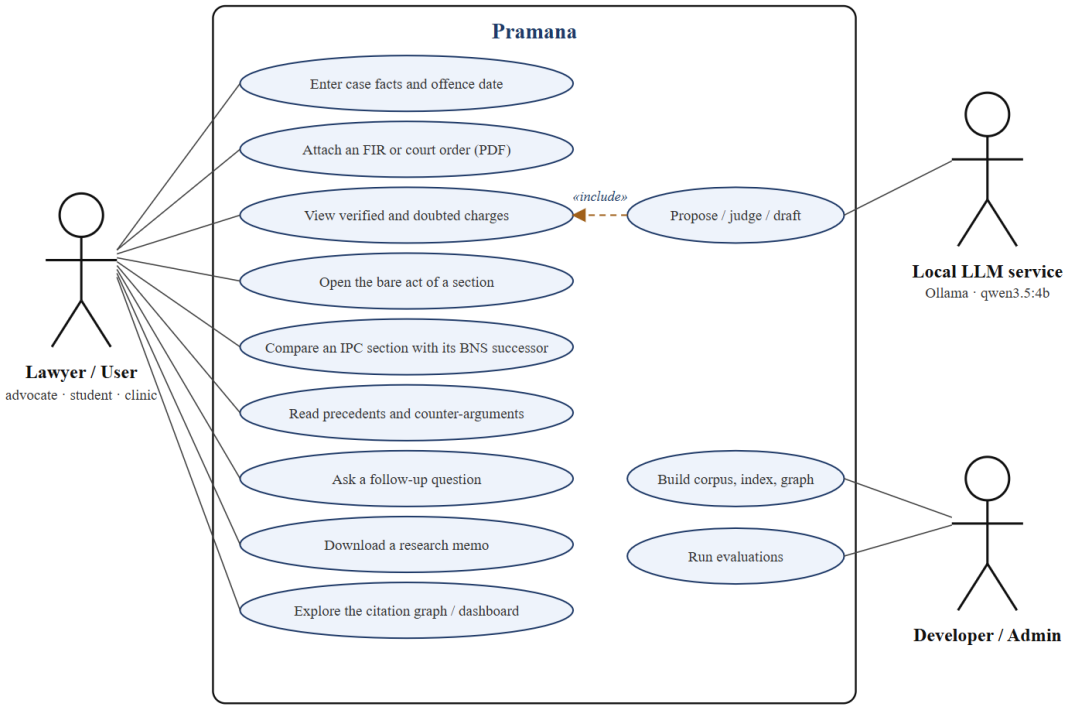


Figure 5: Use case diagram

Actors in the System

1. Lawyer / User: an advocate, law student or legal-aid clinic member who submits cases and reads reports.

2. Local LLM service: the language model that proposes sections, judges applicability and drafts arguments.

3. Developer / Administrator: builds the corpus, index and graph, and runs the evaluations.

Use Cases

1. Enter case facts and the offence date.
2. Attach an FIR or court order as a PDF; the alleged facts are extracted first.
3. View verified, doubted and included charges.
4. Open the bare act of any section, with punishment and bail classification.
5. Compare an IPC section with its BNS successor, word by word.
6. Read precedents and counter-arguments.
7. Ask a follow-up question answered only from the report.
8. Download a research memo.

9. Explore the citation graph and the evaluation dashboard.
10. Build the knowledge base and run evaluations (administrator).

14. IMPLEMENTATION DETAILS

14.1 Project Structure

Table 5: Source modules

Module	Responsibility
src/ingest/	statute and judgment parsing, chunking, section extraction, schema validation
src/retrieval/	BGE-M3 embedder, ChromaDB store, rank fusion with precedent votes
src/graph/	citation graph construction and verification functions
src/pipeline/	router, guardrail, LangGraph nodes, legal rules, applicability check, LLM client
src/api/	FastAPI application, report builder, lawyer tools, follow-up assistant
app/web/	browser interface (HTML/JavaScript)
scripts/	build, fetch, evaluation and calibration scripts
tests/	pytest suite mirroring src/ (422 tests)

14.2 Key Implementation Points

- **Constrained decoding:** the model's output follows a JSON schema with bounded lists and string lengths, so it cannot run on indefinitely; a fact profile (number of accused, shared plan, injury grade, weapon, intent) is produced before the section list.
- **Retry prompt design:** the model's previous answer is replayed as an assistant turn and rejected numbers are listed as banned; sending the reason as a plain follow-up message was ignored in testing.
- **Context sizing:** the smallest of 8K, 16K or 32K tokens that holds the prompt is used, because longer prompts would be silently truncated.
- **Ablation switches:** each legal rule and pipeline change can be switched off for measurement.
- **Live progress:** each stage emits start and done events with a summary of what it produced, which the interface uses to animate the pipeline view.

14.3 API Endpoints

Table 6: Main API endpoints

Endpoint	Purpose
POST /analyze	full analysis of facts and an offence date → report
POST /jobs, GET /jobs/{id}	the same, with live stage-by-stage progress
GET /section/{code}/{s}	bare-act text, punishment, classification, counterpart
GET /compare/{code}/{s}	IPC ↔ BNS comparison with a word-level diff
GET /graph/{code}/{s}	citation-graph neighbourhood of a section
POST /ask, POST /summarise	follow-up questions; precedent summaries
POST /pdf	text of an uploaded FIR or order
GET /stats, GET /health	dashboard figures; system status

14.4 Web Interface

The interface has three views: a workspace for cases, a citation-graph explorer and a system dashboard. During a run it shows the pipeline stage by stage, the candidate sections with their scores and a verification ledger. Each report opens with a result summary, followed by the applicable sections, precedents, counter-arguments and outcome statistics, and an "Explore the evidence" section with the citation graph, an evidence map and an embedding map (Figures 6 and 7).

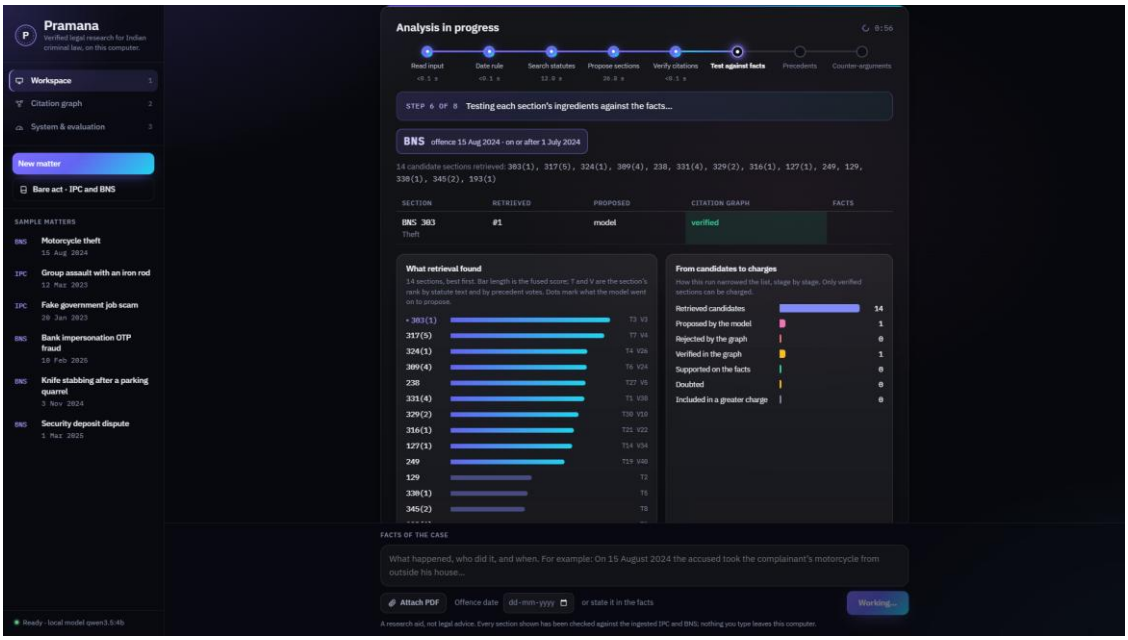


Figure 6: Live pipeline view during an analysis

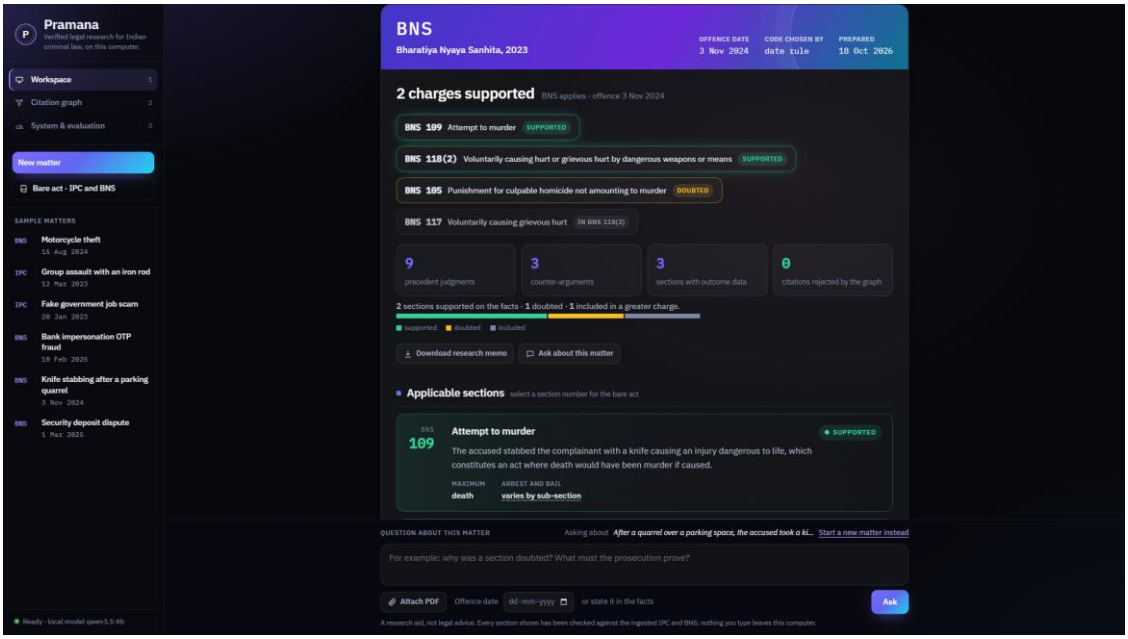


Figure 7: Report with supported, doubted and included charges

15. TESTING & EVALUATION

Overview

Testing ensured that each component works correctly in isolation and that the full system meets its objectives on independent data.

1. Testing Approach

a) Unit testing: each module (ingestion, section extraction, retrieval, graph, router, nodes, legal rules, API) has its own tests; 422 automated tests pass.

b) Integration testing: the pipeline is tested with fake embedders and models so that state transitions, the retry edge and the parallel branches are checked deterministically.

c) System testing: complete runs on test sets, real court orders and the benchmark, plus a click-through test of the web interface.

2. Test Sets

Table 7: Evaluation sets

Test	Set	How it was run
Citation gate	10 real + 10 fabricated numbers	verify_section on each, both codes
Routing	10 cases × 3 runs	code chosen and every verified section's code
Stress test	15 development cases	retrieval withheld; the model cites from memory
Real court orders	30 fresh Patna High Court orders (CC-BY-4.0)	never read before scoring
IL-TUR LSI	100 dev + 100 test documents	decisions on dev; test run once at the end
Embedder bake-off	15 hand-written cases	each searched only against its own code
Held-out sets	16 · 10 · 12 cases	written before each change, scored once

3. Evaluation Metrics

a) Retrieval recall@k: whether a relevant section appears among the top k candidates.

$$Recall@k = (cases\ with\ a\ relevant\ section\ in\ the\ top\ k) / (total\ cases)$$

b) Hallucination catch rate: the share of the model's proposals rejected by the verifier, and whether affected cases recover.

$$Catch\ rate = rejected\ proposals / total\ proposals$$

c) Statute identification (IL-TUR): precision, recall and F1 per label, averaged over the 100 labels.

$$Macro\text{-}F1 = (1 / L) \times \Sigma\, F1(label), \quad F1 = 2PR / (P + R)$$

d) Leakage: the number of rejected or non-existent citations that reached a report; the target is zero.

16. RESULTS

Table 8 summarises the main results; every figure comes from a saved run that can be reproduced with the project's evaluation scripts.

Table 8: Main results

Measure	Result
Invented or wrong-code sections that reached a report, across every evaluation	0
Citation gate: real vs fabricated section numbers	20 / 20
Offence-date routing (10 cases × 3 runs)	30 / 30
Citations checked across 263 saved runs; unverified citations shown	889 checked; 0 shown
Model citing from memory: citations rejected / cases recovered	20 of 68 · 6 / 6
Embedder bake-off, recall@3 (BGE-M3 vs InLegal-Sbert)	0.533 vs 0.200
Real court orders: right section charged (fresh 30)	13 / 30
IL-TUR LSI macro-F1, test split (dev split)	11.8 (9.8)

16.1 Comparison with Existing Work

Table 9: IL-TUR Legal Statute Identification, macro-F1 over 100 labels

System	Training	Macro-F1
LeSICiN	supervised	28.1
Hierarchical InLegalBERT	supervised	26.2
Hierarchical LegalBERT	supervised	21.7
Pramana, test split (100 documents)	none (zero-shot)	11.8
Pramana, dev split (100 documents)	none (zero-shot)	9.8

The supervised systems learn the benchmark's label conventions from 42,000 training documents; Pramana uses no task-specific training. It performs well on serious, well-represented offences (test F1 for IPC 302, murder: 0.78); the gap comes mainly from minor companion offences that rarely reach the candidate list, and from extra sections that exist but do not apply.

16.2 Worked Example

For the motorcycle-theft case (offence date 15 August 2024) the router selected the BNS; retrieval ranked BNS 303 first of 14 candidates; the identifier proposed BNS 303 and the verifier confirmed it; the judge supported it on both passes; the precedent matcher returned three IPC 379 judgments from the Telangana, Himachal Pradesh and Delhi High Courts; the outcome analyst reported IPC 379's 158 judgments; and the drafter produced one argument, that dishonest intention is not proved by unauthorised movement alone, with its quotation verified. The run took about 71 seconds end to end.

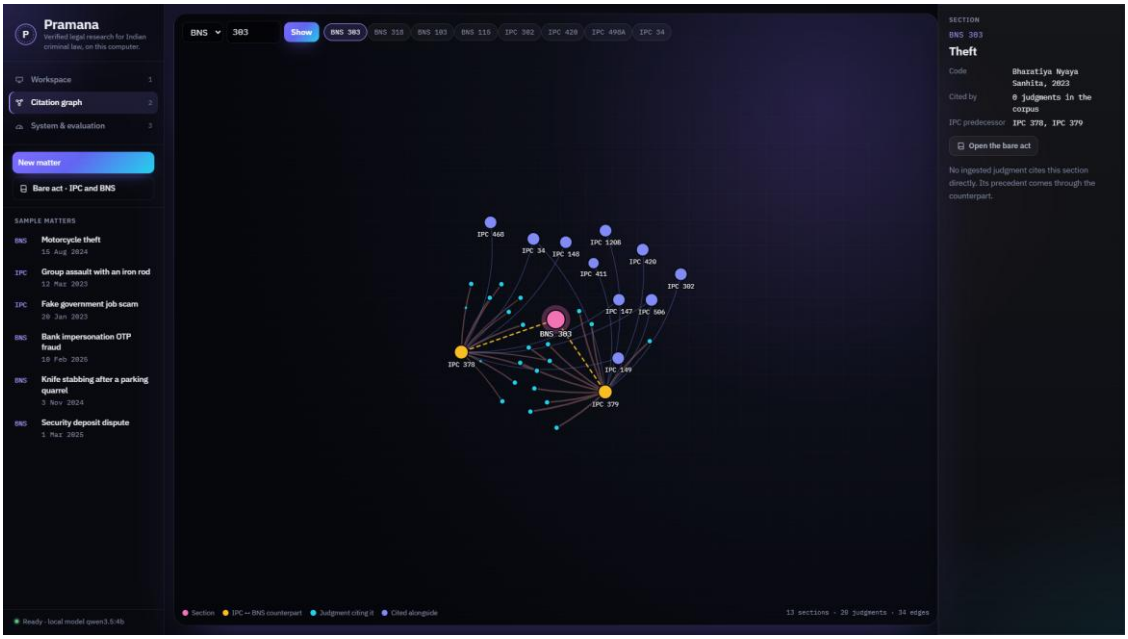


Figure 8: Citation-graph view around BNS 303 and its IPC predecessors

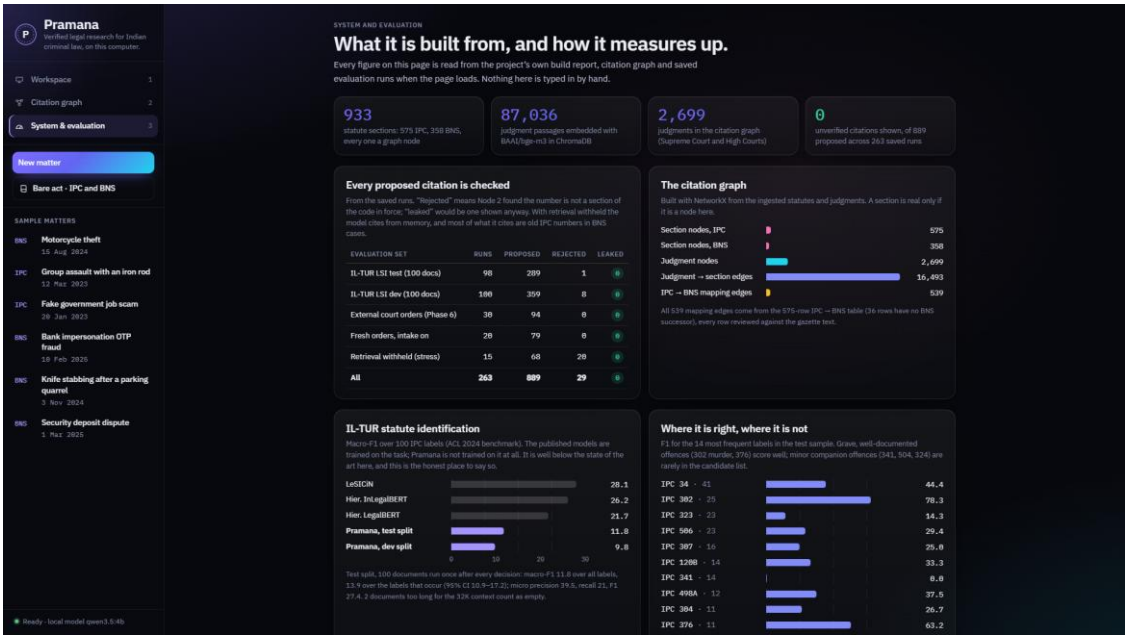


Figure 9: System and evaluation dashboard

17. LIMITATIONS

- **Section choice:** the model sometimes proposes extra sections that exist but do not apply; the applicability check marks many of them as doubted, not all.
- **Benchmark gap:** zero-shot statute identification trails supervised systems on IL-TUR (11.8 against 28.1 macro-F1).
- **Minor companion offences:** offences such as simple hurt or intentional insult are often missed, because only 37% of the right sections reached the candidate list on the test split.
- **Scarce BNS case law:** post-2024 precedent comes almost entirely through IPC predecessors.
- **Review:** the IPC→BNS mapping and the hand-written test keys were prepared with AI assistance and have not yet been reviewed by a practising lawyer.
- **Analytics:** outcome figures are appeal disposals, not conviction rates, and sentence ranges are not extracted.
- **Repeatability:** the same facts can occasionally give a different extra section on a second run.
- **Scope:** substantive criminal law only, in English text; scanned documents without a text layer are not read.

18. FUTURE SCOPE

- **Wider retrieval** for minor companion offences, the largest source of missed sections.
- **Lawyer review** of the IPC→BNS mapping and the legal rules by practising advocates.
- **New BNS judgments** ingested as the higher courts publish them, reducing reliance on the IPC bridge.
- **A precision model** trained on (facts, section) pairs to reduce over-charging.
- **Prior-case retrieval evaluation** on IL-TUR's PCR task, and broader High Court coverage.
- **Procedure and evidence codes:** extension to the BNSS and the Bharatiya Sakshya Adhiniyam.
- **Hindi input and OCR** for scanned FIRs and court orders.

19. CONCLUSION

Pramana shows that an AI legal research assistant can make citation existence a guaranteed property rather than a hoped-for one, even across a wholesale renumbering of a country's criminal code. It does so by routing the code deterministically from the offence date, verifying every proposed section against a code-scoped citation graph, bridging new BNS charges to old IPC precedent, and confining the language model to proposing, judging and drafting, with each of its outputs checked again by code.

The evaluation shows both the strength and the remaining challenge with real numbers: no invented or wrong-code section reached a report in any evaluation, routing was correct every time, and every case recovered when the model cited from memory; choosing exactly the right sections, measured on the IL-TUR benchmark, is the open problem. The project delivers a working multi-agent system, a live web interface, a reproducible evaluation record and a foundation for the future work outlined above.

20. REFERENCES

1. Government of India, “The Bharatiya Nyaya Sanhita, 2023 (Act No. 45 of 2023),” Gazette of India, Ministry of Law and Justice, 2023.
2. Government of India, “The Indian Penal Code, 1860 (Act No. 45 of 1860).”
3. P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Proc. NeurIPS, 2020.
4. S. Paul, P. Goyal and S. Ghosh, “LeSICiN: A heterogeneous graph-based approach for automatic legal statute identification from Indian legal documents,” in Proc. AAAI, 2022.
5. S. Paul, A. Mandal, P. Goyal and S. Ghosh, “Pre-trained language models for the legal domain: A case study on Indian law,” in Proc. ICAIL, 2023.
6. A. Joshi, S. Paul, A. Sharma, P. Goyal, S. Ghosh and A. Modi, “IL-TUR: Benchmark for Indian legal text understanding and reasoning,” in Proc. ACL, 2024.
7. S. Paul, R. Dogra, P. Goyal and S. Ghosh, “ILSIC: Corpora for identifying Indian legal statutes from queries by laypeople,” in Findings of EACL, 2026.
8. S. K. Nigam et al., “NyayaAnumana and INLegalLlama: The largest Indian legal judgment prediction dataset and specialized language model,” in Proc. COLING, 2025.
9. J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian and Z. Liu, “BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation,” arXiv:2402.03216, 2024.
10. D. Premasiri, T. Ranasinghe and R. Mitkov, “LLM-based embedders for prior case retrieval,” in Proc. RANLP, 2025.
11. G. V. Cormack, C. L. A. Clarke and S. Büttcher, “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods,” in Proc. SIGIR, 2009.
12. M. Dahl, V. Magesh, M. Suzgun and D. E. Ho, “Large legal fictions: Profiling legal hallucinations in large language models,” Journal of Legal Analysis, vol. 16, no. 1, 2024.
13. V. Magesh, F. Surani, M. Dahl, M. Suzgun, C. D. Manning and D. E. Ho, “Hallucination-free? Assessing the reliability of leading AI legal research tools,” arXiv:2405.20362, 2024.
14. Qwen Team, “Qwen3 technical report,” arXiv:2505.09388, 2025.
15. LangChain, “LangGraph,” https://github.com/langchain-ai/langgraph; Ollama, https://ollama.com; Chroma, https://www.trychroma.com, 2026.